



CP2106 [ORBITAL]

Team Members (Student ID):

Minchai Kwak (A0288253W)

Lam Kelsey Kylie (A0288477E)

Team Name:

Dimension Hoppers

Team Number:

6074

Table of Contents

Project Overview.....	4
Documentation.....	4
Project Description.....	4
Target Audience.....	4
Motivation.....	4
Tech Stack.....	4
User Stories.....	5
Setting.....	5
Features.....	6
Main Menu.....	6
2D Card Game.....	6
Scriptable Objects.....	6
Card Deck Management System.....	7
Enemy Behaviour Management System.....	7
Player Management System.....	8
3D Fighting Game.....	9
Player.....	10
<Movement via Keyboard Control>.....	10
<Mouse Control>.....	10
<Attack via Keyboard Control>.....	11
<Health System of Player >.....	12
The Egg.....	14
<Enemy AI>.....	14
Dragon.....	15
<Animation States of Enemy AI>.....	15
Summary.....	16
Dimension Hopping.....	16
PlayerPref for Data Transferability.....	17
Software Engineering Principles.....	18
Object-Oriented Programming.....	18
Open-Closed Principle.....	18
Separation of Concerns Principle.....	18
Law of Demeter.....	19
Software Design Patterns.....	20
Singleton Pattern.....	20
Observer Pattern.....	21
State Pattern.....	22
Component Pattern.....	23
Software Project Management.....	24

Project Board (on Github/Trello).....	24
Revision Control.....	24
Project Planning.....	24
Software Development Lifecycle.....	25
Software Design Testing.....	26
Unit Testing.....	26
Appendix - Flow of Gameplay.....	32
References.....	36

Project Overview

Team Name: Dimension Hoppers

Team Number: 6074

Members: Lam Kelsey Kylie (A0288477E), Minchai Kwak (A0288253W)

Project Name: Dimension Hoppers

Level of Achievement: Gemini

Estimated Gameplay Time: 10 - 20 minutes per game

Documentation

Github: <https://github.com/KwakMinchae/Dimension-Hoppers-new>

Dimension Hoppers Build:

https://drive.google.com/file/d/1KhR9IkIx_QHEXvoL-WlqtBJHt-IcWoKy/view?usp=sharing

Project Description

Dimension Hoppers is a unique dimension-hopping fighting game that utilizes both 2D systems and 3D systems by integrating a strategy card game and a 3D first-person melee-based fighting game into a single fast-paced and replayable game.

Target Audience

Dimension Hoppers aims to appeal to esports players by creating a new game genre and gamers who seek intellectually stimulating gameplay.

Motivation

As both team members are long-term esports fans, we have noticed a trend in esports games which are widely enjoyed and available in the market that they tend to fall into a few genres: MOBAs, FPS games and fighting games. Currently, there are few games that utilize and combine both a high intellectual level (for example, strategic card games such as Hearthstone or real-time strategy games such as StarCraft, both of which have become rather niche esports game genres nowadays) and high reaction times (for instance, in fighting games or shooting games). Moreover, few games currently utilize 2D and 3D within the same game.

This game idea initially came from one of two members a few years ago, but it never came to fruition due to a lack of motivation and skills to truly create such a game. Therefore, we want to create a new type of game and inspire a new genre of games, a fusion of relatively unrelated and declining esports game genres.

Tech Stack

- For Game Creation: Unity and scripting in C#
- For Version Control: Git and Github

- Graphics Design: Krita

User Stories

- As gamers who wants to play unique games that are not readily or commonly available in the market, I want to be able to create a new experience that provides a breath of fresh air to the oversaturated multiplayer fighting games market.
- As young adults or teenagers who play relatively more violent video games, I want to use this newly created game to prove that there are benefits to intellectual development from playing such games by adding another genre to fighting games.
- As viewers or fans of complex and diverse sports games such as chessboxing, I want to replicate the feeling of needing to plan and balance strategies between two relatively different tasks.
- As esports viewers who are getting bored of the mainstream esports games that host competitions, I want to create a new game geared towards the esports market and potentially become the next esports game that attracts a large player/viewer base.

Setting

Dimension Hoppers takes place during medieval times when close-distance combat using weaponry such as swords and shields was the best technology one could use in a fight. Dueling was a very popular sport that many enjoy partaking in. However, in order to prevent unnecessary deaths, a group of duelists came up with a variant of dueling named “saut dimensionnel” which translates to “dimension hop”. This version of dueling introduced a card game aspect into the duel between rounds of physical bouts and therefore helped reduce the number of casualties greatly.

Features

Main Menu



Figure 1a & 1b: Main Menu scene and the script

Clicking on the **PLAY** button will send players to the 2D card game's scene while clicking on the **QUIT** button will exit the game application.

2D Card Game

Scriptable Objects

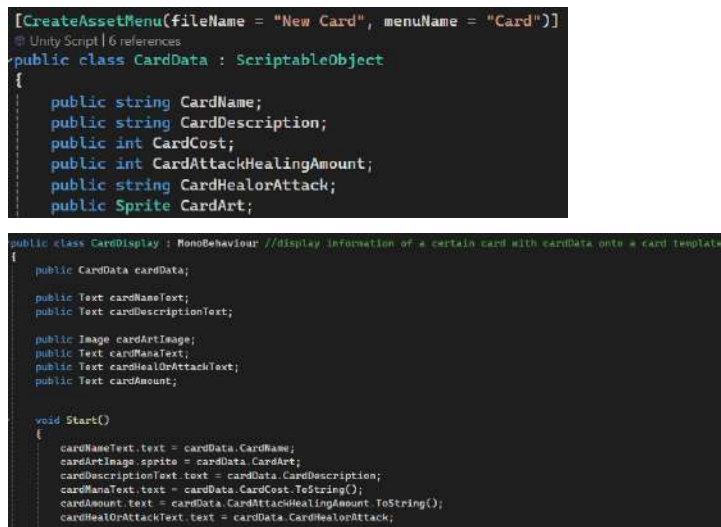


Figure 2a & 2b: CardName and CardData scripts

Scriptable objects are used to store all card variants that are included in the 2D card game portion. Scriptable objects have been chosen as the data storage method as this method promotes reusability of code and has better performance than reading data from a traditional external database tool.

The CardData script includes all fields entered into a new Card and the CardDisplay script is attached to a template card to display all required card information to the player.

Card Deck Management System

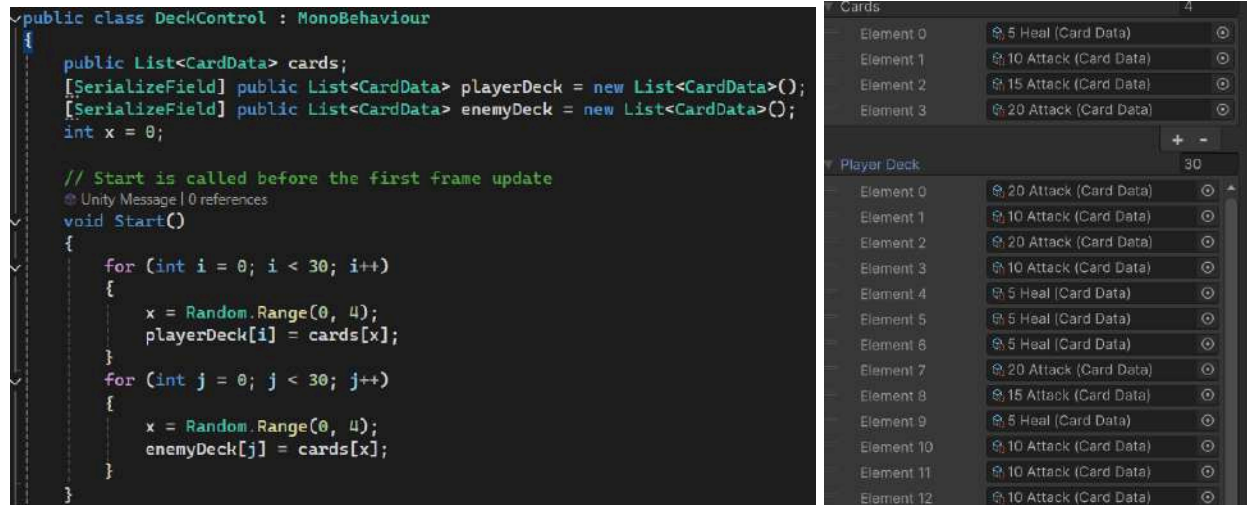


Figure 3a & 3b: Deck Control script and the Inspector

The card deck is generated randomly from all available card scriptable objects and stored as a list. Two separate different lists of available cards will be generated when the scene loads: One player deck and one enemy deck.

Enemy Behaviour Management System

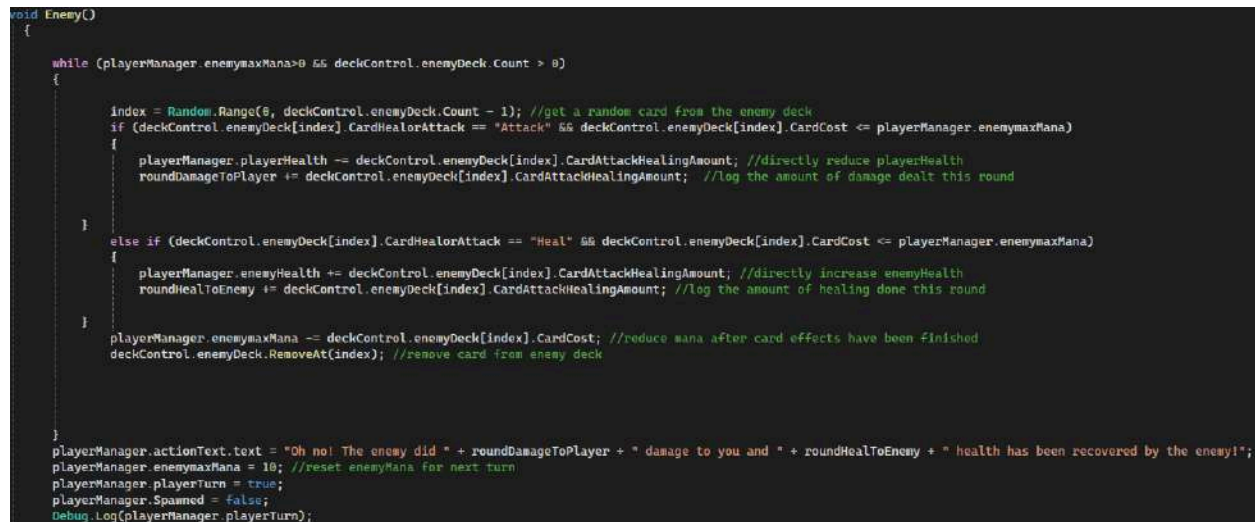


Figure 4: Enemy Deck Control

Currently, the enemy behaviour is defined with the rule that cards are played in order of what exists in the enemy deck list and cards continue to be played as long as it does not exceed the maximum mana amount. Enemy behaviours are collected into a cumulative value and shown to users at the end of the enemy turn on a text box on screen.

Player Management System

```
private void FixedUpdate()
{
    if (playerTurn == true && gameCycle == 0) //check if it is player's turn
    {
        for (int i = 0; i < 5; i++) //initial spawn five cards
        {
            CardSpawn();
        }
        DimensionHopCardSpawn(); //spawn the DimensionHopCard if it is a new load of 2D
        gameCycle++;
        Debug.Log("Initialize Gamecycle: " + gameCycle);
        Spawned = true;
    }
    else if (playerTurn == true && Spawned == false)
    {
        for (int i = 0; i < 5; i++) //spawn five cards at once to "refresh" the board of cards
        {
            CardSpawn();
        }

        gameCycle++;
        manaAmount = 10; //resetting player mana amount
        Spawned = true;
        Debug.Log("Gamecycle: " + gameCycle);
    }

    if (gameCycle == 4) //dimensionhop scenario reached
    {
        Debug.Log("Gamecycle: " + gameCycle);
        SceneManager.LoadScene("3D");
    }
}

void CardSpawn()
{
    GameObject card = Instantiate(Resources.Load(deckControl.playerDeck[index].CardNickname, typeof(GameObject))) as GameObject;
    card.transform.position = nextPos[posAvailable];
    posAvailable++;
    index++;
    if (posAvailable > 4)
    {
        posAvailable = 0; //loop through the five existing positions to do a full "refresh" of cards on hand each turn
    }
    if (index >= 30)
    {
        index = 0; //ensure there will not be out of range of index
    }
}

//reference
void DimensionHopCardSpawn()
{
    GameObject dimensionHopCard = Instantiate(Resources.Load("Dimension Hop", typeof(GameObject))) as GameObject; //spawn dimension
    dimensionHopCard.transform.position = new Vector3(700, 0, 1000); //fixed position for dimensionHop card as it can only spawn e
```

Figure 5a & 5b: Player Deck Control

The playerManager script includes the card spawning function and tracks changes to player health. Currently, it is set so that five new card spawns on screen for the players when it is their turn and the dimensionHop card only spawns once per load of the 2D scene. Moreover, this tracks changes to player turns through pressing the space bar and checks for game over scenarios if either player or enemy has zero or negative health. The playerManager also checks for the game's cycle number in order to dimension hop every 3 cycles.


```

private void OnMouseDown()
{
    if (cardData.CardHealerAttack == "Heal" && playerManager.manaAmount >= cardData.CardCost)
    {
        playerManager.playerHealth += cardData.CardAttackHealingAmount; //playerHealth increases
        playerManager.manaAmount -= cardData.CardCost; //manaAmount decreases
        AudioManager.Instance.Play(popSoundEffect);
        playerManager.actionText.text = "Nice! A potion! You healed " + cardData.CardAttackHealingAmount + " health!";
        Destroy(cardPrefab);
    }
    else if (cardData.CardHealerAttack == "Attack" && playerManager.manaAmount >= cardData.CardCost)
    {
        playerManager.enemyHealth -= cardData.CardAttackHealingAmount; //enemyHealth decreases
        playerManager.manaAmount -= cardData.CardCost; //manaAmount decreases
        AudioManager.Instance.Play(popSoundEffect);
        playerManager.actionText.text = "Great job! You swung your " + cardData.CardName + " at the enemy and dealt " + cardData.CardAttackHealingAmount + " damage to the enemy!";
        Destroy(cardPrefab);
    }
    else
    {
        turnEnd = true; //no more or not a possible move
    }

    Debug.Log("Enemy health is " + playerManager.enemyHealth);
    Debug.Log("Player health is " + playerManager.playerHealth);
}

```

Figure 5c: Card Destroy Control

The CardDestroyer script holds the information of what happens when cards should be destroyed. When players click on a card, it will disappear and the relevant damage will be decreased from enemy health or increase player health.

```

public class DimensionHopCard : MonoBehaviour //specifically attached to the Dimension Hop Card
{
    public AudioClip popSoundEffect;
    [SerializeField] GameObject dimensionHopCard;
    // Unity Message | 0 references
    private void OnMouseDown() //when pressed by mouse, go to 3D scene
    {
        AudioManager.Instance.Play(popSoundEffect);
        SceneManager.LoadScene("3D");
        Destroy(dimensionHopCard);
    }
}

```

Figure 5c: DimensionHop Card Control

The DimensionHop script acts similarly to the CardDestroyer script, but it is tailored specifically for the dimensionHop card so that the scene will be changed to "3D" immediately upon click of the card.

3D Fighting Game

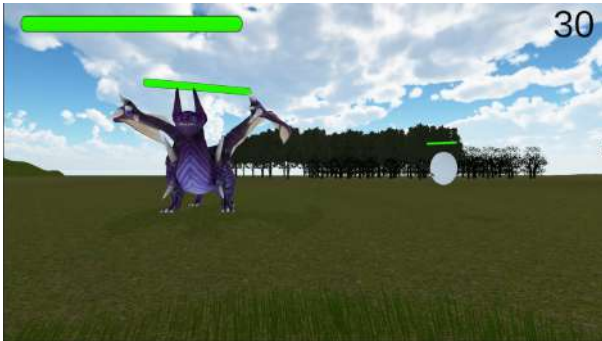


Figure 6: Game View of 3D scene

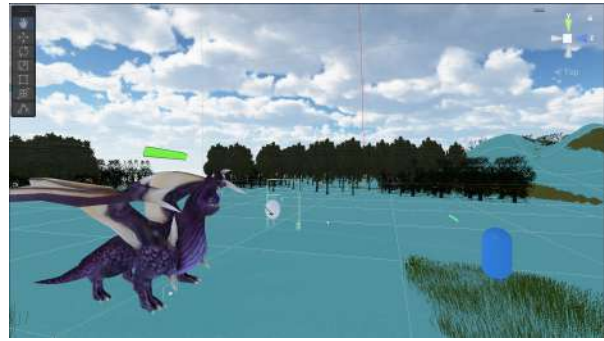


Figure 7: Scene view of 3D Scene

In the 3D scene, there are three characters - a player, an egg and a dragon enemy. All the three characters are able to execute three types of tasks, which are Moving, Attacking and getting damaged.

A total of 15 scripts were used in the 3D scene. However, there are multiple overlaps between these scripts as the three characters all have the same tasks in common. Hence, this documentation will only explain a few scripts and some unity components to avoid redundancy. This section will be divided into the three characters - player, egg and the dragon. Overall, since a set of similar and separate scriptings were used, the project shows a Component pattern.

Player

<Movement via Keyboard Control>

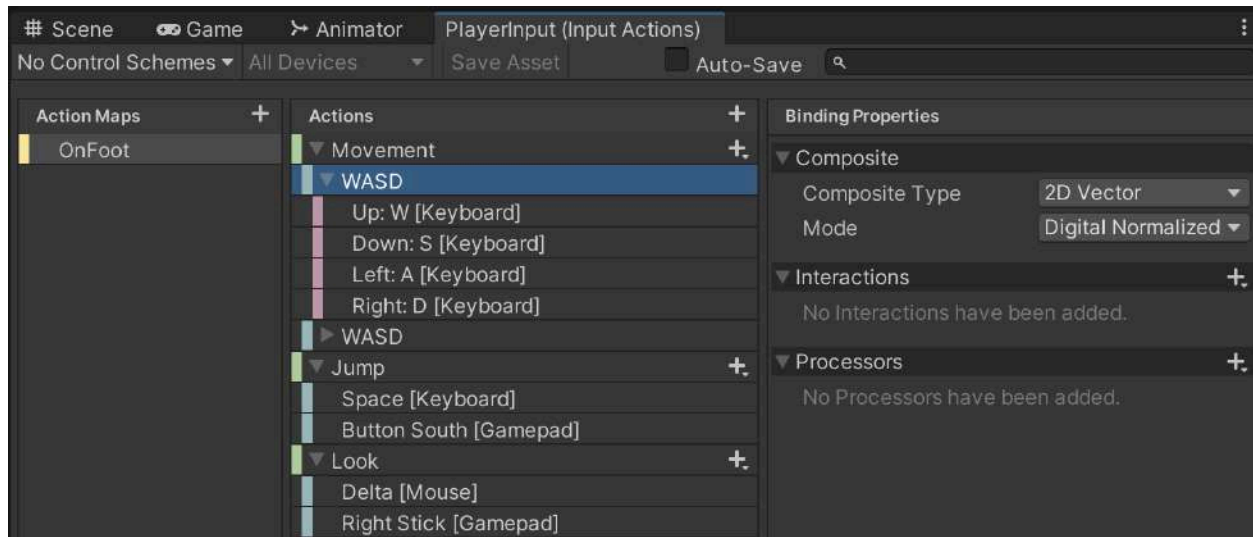


Figure 8: PlayerInput commands

In the Input Actions, different keyboard commands are assigned with movement definitions. The user has the option to play using a gamepad. However, only the keyboard and the mouse were able to be tested in the process of developing the game. The user can move in the environment using WASD commands. To change the direction, a mouse control is implemented to ensure a more intuitive user control whilst playing. The specific script for this function is shown below.

<Mouse Control>

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class PlayerLook : MonoBehaviour
6 {
7     private float x_Rotate = 0f;
8     public float x_Sensitive = 20f;
9     public float y_Sensitive = 20f;
10    public Camera cam;
11
12    public void ProcessLook(Vector2 input)
13    {
14        float mouseX = input.x;
15        float mouseY = input.y;
16        x_Rotate -= (mouseY * Time.deltaTime) * y_Sensitive;
17        x_Rotate = Mathf.Clamp(x_Rotate, -80f, 80f);
18        cam.transform.localRotation = Quaternion.Euler(x_Rotate, 0, 0);
19        transform.Rotate(Vector3.up * (mouseX * Time.deltaTime) * x_Sensitive);
20    }
21 }
```

Figure 9: PlayerLook script

This script enables the camera to follow the mouse movement.

The sensitivity of the camera movement can be adjusted from the `x_Sensitive` and `y_Sensitive` variables.

The degree of rotation is fixed to -80° to 80° .

Quaternion Euler enables smooth camera rotation.

Overall, the range in which the camera rotates and the sensitivity of the movement can be adjusted, but after a series of testing from the developer, the following inputs were deemed suitable.

<Attack via Keyboard Control>

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class WeaponController : MonoBehaviour
6 {
7     public GameObject Sword;
8     public bool CanAttack = true;
9     public float AttackCoolDown = 1.0f;
10    public AudioClip SoundSwordAttack;
11
12    void Update()
13    {
14        if (Input.GetKey(KeyCode.LeftShift) || Input.GetMouseButtonDown(0))
15        {
16            if (CanAttack)
17            {
18                SwordSwing();
19            }
20        }
21    }
22    public void SwordSwing()
23    {
24        CanAttack = false;
25        Animator animator = Sword.GetComponent<Animator>();
26        animator.SetTrigger("Attack");
27        AudioSource audio = GetComponent<AudioSource>();
28        audio.PlayOneShot(SoundSwordAttack);
29        StartCoroutine(AttackCoolDown());
30    }
31    IEnumerator AttackCoolDown()
32    {
33        yield return new WaitForSeconds(AttackCoolDown);
34        CanAttack = true;
35    }
36 }
```

Figure 10: WeaponController script

Initially, the sword and the audioclip are dragged into the Inspector to specify the gameobjects.

The script checks if a leftshift or a left mouse click is inputted every frame. If so, then the sword is swung.

The sword swinging function is defined separately.

The boolean variable CanAttack is set to False within the function. This is to prevent the sword swinging animation to occur simultaneously.

Since the sword is a gameobject, there should be an animation of the sword swinging. The swinging animation was recorded using an animator function on Unity, which was named as 'Attack'. Using the trigger function in the animator, the animation is played when the function is executed. Simultaneously, an audio is played using AudioSource.

The Coroutine and IEnumerator exist so that there is a cooltime of one second between each sword swinging.

<Health System of Player >

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4  using UnityEngine.UI;
5
6  public class PlayerHealthbar : MonoBehaviour
7  {
8      [SerializeField] private Slider slider;
9      [SerializeField] private Camera camera;
10
11     public void UpdatePlayerHealthbar(float currentVal, float maxVal)
12     {
13         slider.value = currentVal / maxVal;
14     }
15     void Update()
16     {
17         transform.rotation = camera.transform.rotation;
18     }
19 }
```

Figure 11: Healthbar of Player

The main purpose of the script is to display the Healthbar UI on the scene.

A namespace of UnityEngine.UI is applied, as the slider UI is used as the healthbar.

The private fields are serialized so that Unity can store and reconstruct later.

The value of the slider is the ratio between the current health and the maximum health, as the health is represented as percentage on the healthbar.

The Update() ensures that the healthbar stays in the fixed position even when the user rotates.

PlayerReceive

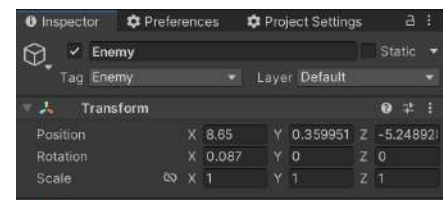
```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class PlayerReceive : MonoBehaviour
6 {
7     public int maxhealth = 500;
8     public int health = 500;
9     public Collider PlayerCollider;
10
11     [SerializeField] PlayerHealthbar Playerhealthbar;
12
13     private void Awake()
14     {
15         Playerhealthbar = GetComponentInChildren<PlayerHealthbar>();
16     }
17     void start()
18     {
19         PlayerCollider = GetComponent<Collider>();
20         PlayerCollider.isTrigger = true;
21         Playerhealthbar.UpdatePlayerHealthbar(health, maxhealth);
22     }
23     private void OnTriggerEnter(Collider other)
24     {
25         if (other.gameObject.tag == "Enemy")
26         {
27             TakeDamage(10);
28         }
29     }
30     public void TakeDamage(int damageAmount)
31     {
32         health -= damageAmount;
33         Playerhealthbar.UpdatePlayerHealthbar(health, maxhealth);
34     }
35 }
```

Figure 12: Health System of Player

The following script is a health system that receives damage from the enemy and updates the player's health accordingly.

Initially, the maximum health of the player is set as 500.

The attack from an enemy is detected using a Box Collider. The tag of the box collider is assigned as "Enemy".



Hence when the enemy attacks the player, the two box colliders will hit, and the tag will be recognised.

Then the TakeDamage() function is implemented to deduct the health of the player and update the healthbar accordingly.

The Egg

<Enemy AI>

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4 using UnityEngine.AI;
5
6 public class EnemyController : MonoBehaviour
7 {
8     public float detectRadius = 10f;
9     Transform target;
10    NavMeshAgent agent;
11
12    void Start()
13    {
14        target = PlayerTracker.instance.player.transform;
15        agent = GetComponent<NavMeshAgent>();
16    }
17
18    void Update()
19    {
20        float distance = Vector3.Distance(target.position, transform.position);
21        if (true) //always
22        {
23            agent.SetDestination(target.position);
24
25            if (distance <= agent.stoppingDistance)
26            {
27                //Attack the target
28                FaceTarget();
29            }
30        }
31    }
32
33    void FaceTarget()
34    {
35        Vector3 direction = (target.position - transform.position).normalized;
36        Quaternion lookRotation = Quaternion.LookRotation(new Vector3(direction.x, 0, direction.z));
37        transform.rotation = Quaternion.Slerp(transform.rotation, lookRotation, Time.deltaTime * 5f);
38    }
39
40    void OnDrawGizmosSelected()
41    {
42        Gizmos.color = Color.red;
43        Gizmos.DrawWireSphere(transform.position, detectRadius);
44    }
45 }
```

Figure 13: Controller of Enemy AI

Previously, the user was able to manipulate player's movements manually. Since this is not a multiplayer game, the enemy is not controlled by any users. It attacks the player 'by itself'. This Enemy Controller script for the egg achieves this purpose.

Essentially, the egg sets its destination as the user's location in all instances.

At the start of the script, the detect radius is 10f, meaning that the enemy initiates chasing when the player is within this radius.

A target is a reference to the player's location.

A NavMeshAgent is used for pathfinding and navigation.

The nested if statement prevents the enemy from attacking the player while facing another direction. Without this condition, the egg swung the sword very close to the player while facing a totally irrelevant direction. If the enemy is not facing the player, the enemy will stop chasing at a stopping distance then face the target. The function to face the player is defined later.

In FaceTarget(), the difference in the direction between the enemy and the player is calculated then normalised.

In Vector3(direction.x, 0, direction.z), the direction of y is set as 0 as the dragon will only have to move in a plane. Quaternion.LookRotation creates a smooth rotation.

OnDrawGizmosSelected() draws a gizmos of red colour for better visualisation, but this is not necessary for the game play.

PlayerTracker

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class PlayerTracker : MonoBehaviour
6  {
7      #region Singleton;
8      public static PlayerTracker instance;
9
10     void Awake()
11     {
12         instance = this;
13     }
14     #endregion
15
16     public GameObject player;
17 }
```

Figure 14: Singleton Player Tracker

In this script, a Singleton Pattern was used to track the player's location in the environment. This is explained in more detail in the 'Software Design Pattern' section.

Dragon

<Animation States of Enemy AI>



Figure 15: Dragon dead in 3D

For the dragon enemy, only its animations are explained since the health and attacking system are almost identical as aforementioned. Unlike the egg which only has a single animation of sword swinging, the Dragon enemy has a variety of animations and multiple states.

In Unity, the animations are mapped and five different parameters will trigger different animations.

Starting from an Idle State, the dragon can be patrolling or chasing. After chasing it will go onto the Basic Attack. Notice how it is impossible to directly go from patrolling to a basic attacking.

Summary

The table below is the overall summary of all the scripts used in the 3D scene.

Summary of the fifteen scripts with brief explanations						
	Player		The Egg		Dragon	
Move	- InputManager - PlayerMotor - PlayerLook	Move using WASD Camera follows mouse	- EnemyController - PlayerTracker	Always faces and follows the player	- AttackState - ChaseState - IdleState - PatrolState	State Machine Behaviours for Enemy AI
Attack Control	- WeaponController	Left shift / Left mouse to swing sword	- EnemyWeapon Control	Always swinging the sword		
Health	- PlayerHealthbar - PlayerReceive	Detects box collider	- PlayerHealthbar - EnemyReceive	Detects box collider	- PlayerHealthbar - DragonReceive	Detects box collider
Timer	Timer					

Figure 16: Features organised for all three characters.

Dimension Hopping

```

using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using TMPro;
using UnityEngine.SceneManagement;

public class Timer : MonoBehaviour
{
    [SerializeField] TextMeshProUGUI timerText;
    [SerializeField] float remainingTime;
    [SerializeField] string sceneName = "> scene name goes here <";
    [SerializeField] private Camera camera;

    // Update is called once per frame
    void Update()
    {
        if (remainingTime <= 0)
        {
            SceneManager.LoadScene( sceneName );
        }

        remainingTime -= Time.deltaTime;
        int minutes = Mathf.FloorToInt(remainingTime / 60);
        int seconds = Mathf.FloorToInt(remainingTime % 60);
        timerText.text = remainingTime.ToString("f0");
        //timerText.text = string.Format("{0:00}:{1:00}", minutes, seconds);

        transform.rotation = camera.transform.rotation;
    }
}

```

```

private void OnMouseDown() //when pressed by m
{
    AudioManager.Instance.Play(popSoundEffect);
    SceneManager.LoadScene("3D");
    Destroy(dimensionHopCard);
}

```

Figure 16a & 16b: Scene change from 3D to 2D & Scene change from 2D to 3D

Players will switch between the 2D card game and 3D fighting game during one match. The criteria for dimension switching is finishing 3 rounds or when the DimensionHopCard is clicked for the 2D card game as seen in the PlayerManager and DimensionHopCard script and when 30 seconds have passed for the 3D fighting game as seen in the Timer script.

PlayerPref for Data Transferability

```
//PLAYERMANAGER in 2D

void Start()
-> LoadHealth();

void Update()
-> if timesinceload >= 30
    SaveHealth();

void SaveHealth()
-> PlayerPrefs.SetInt(PlayerHealth)
-> PlayerPrefs.SetInt(EnemyHealth)
-> PlayerPrefs.Save()

void LoadHealth()
-> PlayerPrefs.GetInt(PlayerHealth)
-> PlayerPrefs.GetInt(EnemyHealth)

//DragonReceive in 3D

void Start()
-> LoadHealth();

void Update()
-> if timesinceload >= 30
    SaveHealth();

void SaveHealth()
-> PlayerPrefs.SetInt(EnemyHealth)
-> PlayerPrefs.Save()

void LoadHealth()
-> PlayerPrefs.GetInt(EnemyHealth)

//PlayerReceive in 3D

void Start()
-> LoadHealth();

void Update()
-> if timesinceload >= 30
    SaveHealth();

void SaveHealth()
-> PlayerPrefs.SetInt(PlayerHealth)
-> PlayerPrefs.Save()

void LoadHealth()
-> PlayerPrefs.GetInt(PlayerHealth)
```

Figure 17: Pseudocodes of PlayerPrefs for 2D and 3D scenes.

The above pseudocodes show how the playerprefs are integrated into the existing scripts for each of the character's health systems. The health point of the player and the enemy are initially saved with `PlayerPrefs.SetInt()`. The data is later retrieved with `PlayerPrefs.GetInt()`. `PlayerPrefs` are only limited to saving and retrieving data in string, float and integer. Since health points are integer type, `PlayerPrefs` are efficient and relevant for this occasion.

Ending Scenes

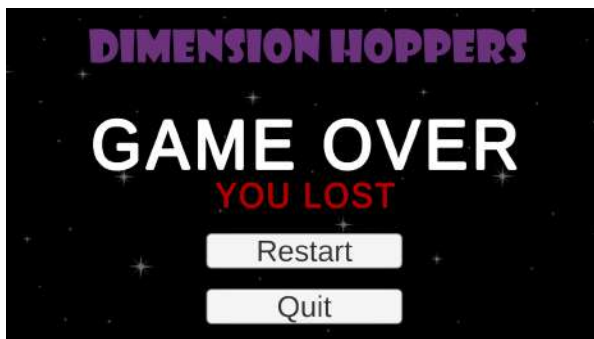


Figure 18a: GameOver Scene

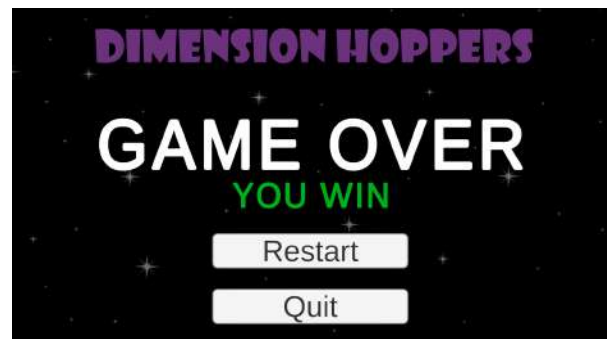


Figure 18b: YouWin Scene

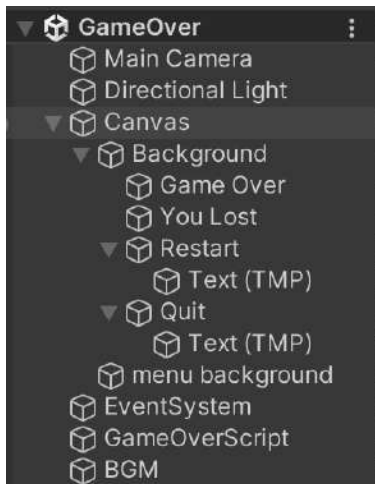


Figure 19: Inspector

When the player's health reaches zero, the player loses the game. When this occurs, the player is loaded to a GameOver scene. Similarly, when the health of the enemy reaches zero earlier than the player's, the player wins the game.

When these scenes are loaded, the player is given two choices - either to Restart or to Quit. These two options are buttons on the same canvas, as shown on the inspector view.

When the player selects 'Restart' button, the "Main Menu" scene is loaded.

```

7  {
8      public void gameRestart()
9      {
10         SceneManager.LoadScene("Main Menu");
11     }

```

The entire application quits when the player clicks on 'quit' button.

```

13     public void quitGame()
14     {
15         Application.Quit();
16     }
17 }

```

Software Engineering Principles

Object-Oriented Programming

Unity uses C# for script writing, which is a object-oriented programming language.

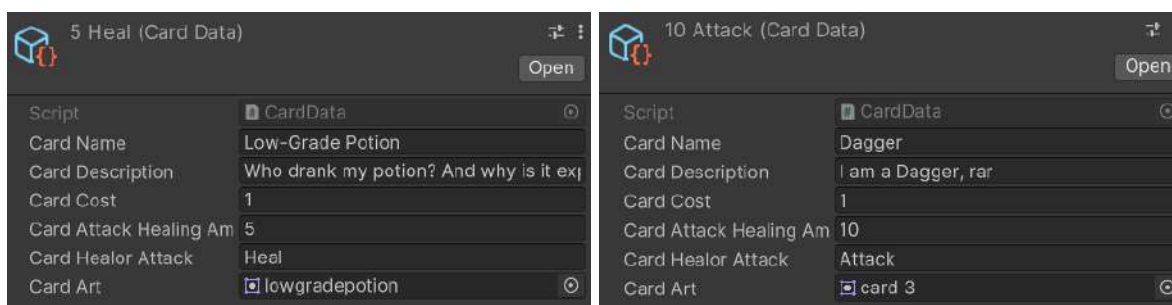
Object-Oriented Programming (OOP) is a software engineering principle that organizes code around objects rather than actions. Each object represents a real-world entity and combines data (attributes) and behavior (methods). Core concepts of OOP include encapsulation (bundling data with methods), inheritance (deriving new classes from existing ones), polymorphism (using a single interface for different data types), and abstraction (simplifying complex systems). OOP enhances code modularity, reusability, and maintainability, making it easier to develop and manage complex software systems.

Single Responsibility Principle

The Single Responsibility Principle states that a class should only do one thing and have only one single reason to change. (CS2103 Textbook, 2021) Dimension Hopper's 2D Card Game's cards Game Objects have multiple functions. The CardDisplay script handles reading and writing information from each card's scriptable objects to the card template within each card prefab. Meanwhile, the CardDestroyer script handles the changing of player and enemy health values and destruction of card objects. Using the Single Responsibility Principle to separate functions into multiple unique scripts allowed us to read and understand what each chunk of code should do or does at a faster speed.

Open-Closed Principle

The Open-Closed Principle aims to make a code entity easy to adapt and reuse without needing to modify the code entity itself. (CS2103 Textbook, 2021) In Dimension Hoppers' 2D Card Game portion, Scriptable Objects are created and used to create and store data required for all card variants in the game as shown in the example images below. The use of scriptable objects adheres to the open-closed principle as only one script is required to create the template of what should be stored in a scriptable object and does not require the need to create a new and separate script for each card type.



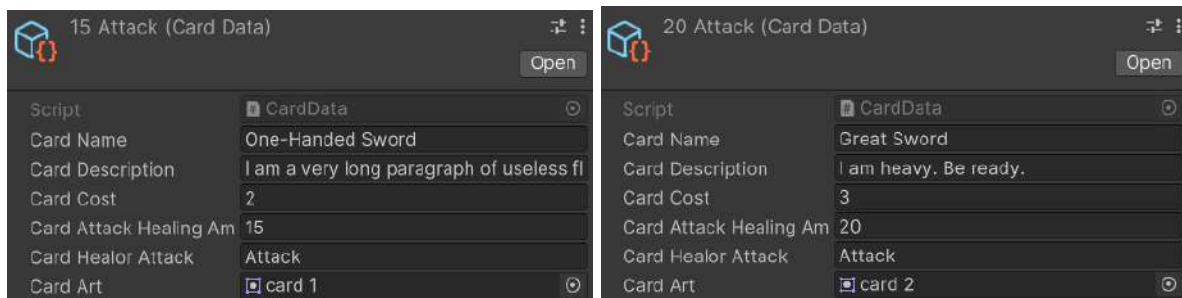


Figure 18: Inspector view of Card Data Attacks

Separation of Concerns Principle

This principle states that the code is separated into distinct sections. This principle is applied to the majority of the scripts. This principle is not only applicable to the level of the individual scripts, but how the overall scripts are used in general. Inside a single script, for example, the EnemyReceive script is able to detect the player's sword with the 'sword' tag. In most of the scripts, the section is divided by Awake(), Start(), Update() and more functions. Each script has a distinct functionality and it does not interfere with another script. The only occasion when it interacts with another script is only when it is essentially needed, such as when the Healthbar UI and Health Damage System scripts interact.

Law of Demeter

This principle is also closely related to the previous principle, Separation of Concerns Principle. The independence between the scripts are essential to avoid the entire system crashing when there is an error in one of the scripts. It makes the recovery much quicker and manageable. Similarly, clearly dividing up the task makes accessing the information from one script much easier. The method of the script will only invoke a certain behaviour. This is shown as the dragon enemy has four different states, although the script for each state is short. Law of Demeter is followed so that one animation is executed at one time only.

Interface Segregation Principle

This principle states that a script should not be dependent on methods that it does not use. The purpose of following this principle is to decrease the dependency across scripts, which is necessary for subsequent refactoring or updates in the future. If a modification in a script requires another compulsory modification in another script, this process is not efficient. For instance, pay attention to the script that contains the PlayerPref of the health for both the enemy and the player.

```
//DragonReceive in 3D

void Start()
-> LoadHealth();

void Update()
-> if timesinceLoad >= 30
    SaveHealth();

void SaveHealth()
-> PlayerPrefs.SetInt(EnemyHealth)
-> PlayerPrefs.Save()

void LoadHealth()
-> PlayerPrefs.GetInt(EnemyHealth)
```

Figure 17. Pseudocode

Including the Start() and Update() methods, there are four methods that are used in total.

Overall, the DragonReceive scripts calculates the damage dealt by the player as part of its health system. Furthermore, this health is saved at the end of the 3D scene, and it is also retrieved from the 2D scene at the start of the 3D scene. In other words, it is a comprehensive health system that keeps track of the Dragon's health throughout the game play.

Although the functionality is complex, the methods are concise and are all necessary. All the methods that are being used are needed for the dragon in the game play.

Hence, this is an example of Integration Segregation Principle being applied.

Software Design Patterns

Singleton Pattern

Overall, the singleton pattern is used to ensure that there is only one instance of PlayerTracker in the game scene.

In the PlayerTracker script, a singleton pattern was implemented for the player's location.

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class PlayerTracker : MonoBehaviour
6  {
7      #region Singleton;
8      public static PlayerTracker instance;
9
10     void Awake()
11     {
12         instance = this;
13     }
14     #endregion
15
16     public GameObject player;
17 }
```

Figure 19: Singleton pattern of Player's instance

The Singleton region is defined by #region Singleton and #endregion.

```
7      #region Singleton;
```

```
14     #endregion
```

Static variable of the player is declared, defined as 'instance'.

```
8      public static PlayerTracker instance;
```

Within the Awake() function, the instance variable is assigned with the instance of the player. With this, the player's instance becomes globally accessible using PlayerTracker.instance.

```
12         instance = this;
```

Then afterwards, this instance is assigned to a GameObject known as 'player', which is the user itself.

```
16     public GameObject player;
```

For the enemies to attack the player, the instance of the player needs to be globally accessible. Hence, any script access PlayerTracker.instance.

It is recognised that there are certain risks associated with using Singleton. However, considering that there is only a single player in this project, implementing the Singleton Pattern

is the most appropriate.

Observer Pattern

In this design pattern, there is a relationship between one and multiple game objects. In other words, one subject influences multiple observers. Observers are notified of the changes made in the subject and these are accordingly updated at changed. This software design pattern is observable specifically in the 3D fighting game's health system and player card handling after input in the 2D card game.

For the 3D fighting game:

1. It initially starts with how the enemy tag is detected in the box colliders.

```
23     private void OnTriggerEnter(Collider other)
24     {
25         if (other.gameObject.tag == "Enemy")
26         {
27             TakeDamage(10);
28         }
29     }
```

2. This detection decreases the overall health of the player by 10 points in the health system. (adjustable)

```
30     public void TakeDamage(int damageAmount)
31     {
32         health -= damageAmount;
33         PlayerHealthbar.UpdatePlayerHealthbar(health, maxhealth);
34     }
```

3. The healthbar UI reflects the new updates in the health.

```
11     public void UpdatePlayerHealthbar(float currentVal, float maxVal)
12     {
13         slider.value = currentVal / maxVal;
14     }
```

In this case, the observer is the tag detection. The observers are the health point and healthbar UI on the gameview.

On a similar note, the mouse control is an example of an observer pattern. Depending on the mouse movement (subject), the Healthbar UI and Timer UI are equally affected (observers).

```
15     void Update()
16     {
17         transform.rotation = camera.transform.rotation;
18     }
```

For the 2D card game:

1. When the system detects a left mouse click on any card game object, the system checks if the player has enough mana to play the clicked card. Upon satisfying the condition, the player health, enemy health and mana amount are changed in the player manager game object.


```
private void OnMouseDown()
{
    if (cardData.CardHealOrAttack == "Heal" && playerManager.manaAmount >= cardData.CardCost)
    {
        playerManager.playerHealth += cardData.CardAttackHealingAmount;
        playerManager.manaAmount -= cardData.CardCost;
        Destroy(cardPrefab);
    }
}
```

- As the values are updated upon the clicking of a card game object, the updated health and mana values are passed to the player manager, which is parsed to the player's screen and displayed.

```
void Update()
{
    manaText.text = manaAmount.ToString();
    playerHealthText.text = playerHealth.ToString();
    enemyHealthText.text = enemyHealth.ToString();
}
```

Overall, the observers react to the changes in the subjects. The main benefit of this system is that the observers receive the latest data while also being able to add more observers. There is less limitations into adding more objects or features into the game.

State Pattern

In state pattern, the behaviour of the object is updated depending on the internal state. This is seen with the how the dragon eventually dies when its health reaches zero. The gradual decline in health is not reflected in the most recent prototype. Only when the dragon is fully damaged, the dead state will be shown. As long as the health is greater than zero, the dragon is able to stand, patrol, walk, run or attack. However, these five states are not affected by level of health. This can be seen as a Finite State Machine as only one state is possible at a time. This is logically understandable since it is impossible for the dragon to be dead and to be attacking simultaneously.

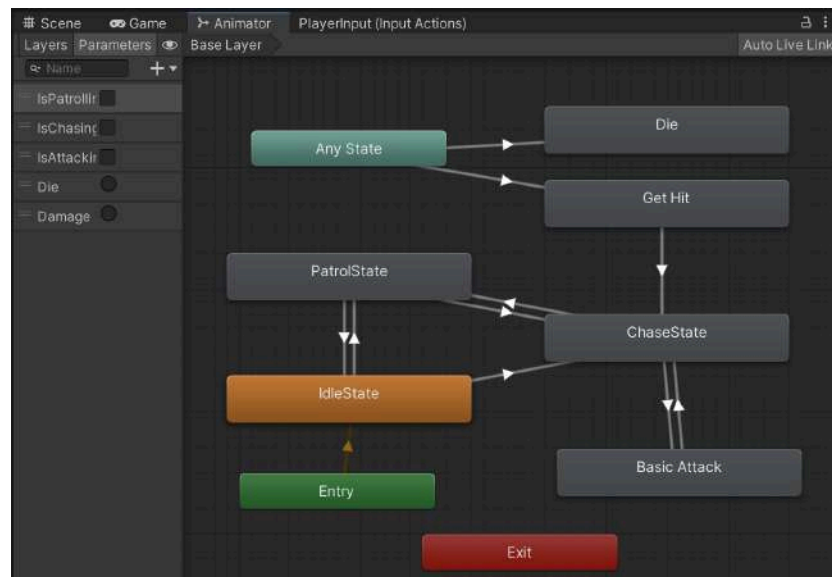


Figure 20: Animator State Pattern

Component Pattern

This design pattern highlights the independence of functionality in the game objects. This pattern is heavily used for all of the three characters in the game. As aforementioned, there are three characters in 3D. In terms of its movements, health system and attack control, there is a huge amount of overlap. The table below summarises this.

	Player	The Egg	Dragon
Move	WASD key	One state: -Always following player -SetDestination() using Navmesh AI	Four Animation states: • Idle • Patrolling • Chasing • Attacking
Attack Control	Left shift / Left mouse click	One state: -Always swinging sword	Two states: • Attack • Not Attack
Health	Health reduced when enemy's box collider touches player's box collider. (and vice versa) PlayerPref for Data Transferability across scenes.		

Figure 21: Summary of Characters and Features

Hence, scripts such as the Healthbar UI and the health system has been applied to all three characters.

Software Project Management

Project Board

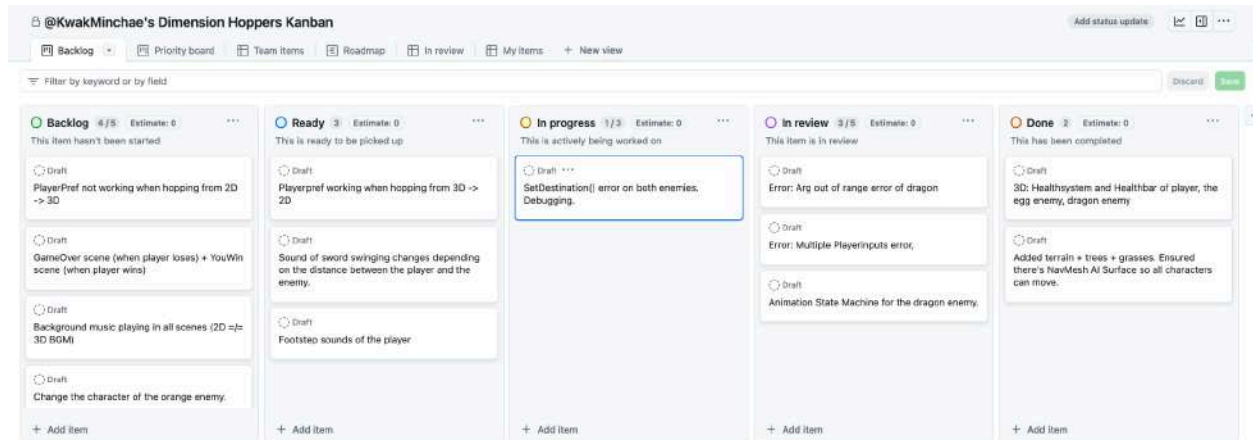


Figure 22: Dimension Hoppers Kanban in the last stages, on Github

The team ensured the consistency of productivity by using Kanban on Github. We have come to the conclusion that working on one issue/error/feature at a time is the most efficient. Focusing on a single feature ensured that we stayed focussed to the goal.

Revision Control

Our team uses Github for revision control. As both team members mostly work on their separate parts' features, code, game files, assets and other project files are committed and pushed onto the repository each time a major feature has been updated or created. Using Github helped us detect and reduce the chance of merge conflicts at an early stage and notified us to change or overwrite certain files to solve such merge conflicts.

Project Planning

At the beginning of each milestone, we had a team meeting detailing what features should be implemented, added or deleted for the current milestone. As we discovered during the second milestone, our project's initial scope was too wide for us to accomplish within the Orbital milestone deadlines as our software engineering experience was too shallow. This led us to revamp the project from a multiplayer game into a single-player game with the same core features (2D card game and 3D fighting game). We mainly used Google Docs to keep track of the features to be completed by each milestone. Below is an example of our detailed planning

for Milestone 3:

ReadmeTodo (21st July) ☐ Read exemplary Readme (mentor) on how he did the testing. ☐ Upload build file on Github (when the thing is stable) ☐ Linking the 2D and 3D components of your game together and playthrough. ☐ Include SWE Principles ☐ SWE Patterns ☐ SWE Practices ☐ Unit Testing ☐ Integration Testing ☐ System Testing ☐ Regression Testing ☐ User Acceptance Testing Refer to CS2103T textbook for the various types of testing. You will have to fulfill minimally the 5 types of testing above for Apollo.	Email (8th July) • Clarify the new level of achievement, and if he still hasn't downgraded for us, ask him to do so. • Clarify (gemini)'s requirements to pass orbital • Can you help us look at our (final draft) of the readme and give comments Timeline 14th July: Playerpref work (HP) – finish the game this wk. 14th + (continuous) Testing and Debugging: Discovering bugs. 24th July: 90% done with Readme 3 29th July: MS3 Deadline
---	--

Figure 23: Plan for Milestone 3

Software Development Lifecycle

Our team mainly used an iterative model to produce our game application. By reviewing and creating a new iteration every milestone during the project, we factor in previous iterations' feedback from both our team's members and feedback received from peers and our mentor in order to adjust the timeline of development, items or features to be developed and estimated effort or time required to achieve certain features. We mainly used the depth-first approach in our development process, where we focused on fleshing out the main features of the game and could present what each feature was supposed to do even though they were not fully developed yet.

Software Design Testing

Integration Testing

This software design testing evaluates when all the components, classes and scripts are all combined. Integration testing has yielded the test cases, which are explained in the subsequent section. The purpose of the testing is to check how the software components interact, even if it might have been written by different programmers. Considering that the two of us independently wrote one scene per person, integration testing is the most suitable to identify errors in communication between components.

Integration Testing is highly significant because one of the main features of the game is dimension hopping between 2D and 3D. To create a more realistic experience, the health of the player and the enemy are preserved. For this to be achievable, PlayerPrefs was used. During the initial stages of the Integration Testing, there were multiple errors regarding PlayerPrefs.

However, we eventually found where the error was occurring in the communication between the scripts across scenes. Hence, the core feature of the game, health perseverance in dimension hopping, works successfully.

Test Cases

Our team mainly used manual testing for our unit testing according to the test cases listed below:

Feature: Main Menu			
Scenario	Test case	Expected Result	Actual Result
On scene load	Play button is pressed	Scene switched to "2D" and 2D card game starts playing	Scene switched to "2D" and 2D card game starts playing
	Quit button is pressed	Exit the game application	Exit the game application

Figure 24: Main Menu test case

Feature: 2D Card Game			
Scenario	Test case	Expected Result	Actual Result
On scene load	When scene is loaded for the first time	A deck of five random cards are seen on the player's screen, and a dimension hop card is seen on the rightmost	A deck of five random cards are seen on the player's screen, and a dimension hop card is seen on the rightmost

		of the player's screen as a sixth card.	of the player's screen as a sixth card.
		The player's and enemy's health is shown as 500. The player mana amount is shown as 10.	The player's and enemy's health is shown as 500. The player mana amount is shown as 10.
		The player has the first turn to make actions.	The player has the first turn to make actions.
	When scene is loaded again	Player health and enemy health is the same as the previous 3D scene.	Player health and enemy health is the same as the previous 3D scene.
		Player and Enemy mana is displayed as 10.	Player and Enemy mana is displayed as 10.
	On all player turns	The enemy's health is deducted by the amount listed on the card, updated enemy health is reflected on the player screen.	The enemy's health is deducted by the amount listed on the card, updated enemy health is reflected on the player screen.
		Player mana amount is deducted by the card's listed card cost, updated remaining mana amount is listed on player screen.	Player mana amount is deducted by the card's listed card cost, updated remaining mana amount is listed on player screen.
		Selected card is destroyed and removed from player screen, card is removed from player deck.	Selected card is destroyed and removed from player screen, card is removed from player deck.
		When an action card (heal type) (excluding dimension hop card) is clicked and current	The player's health is increased by the amount listed on the card, updated player

	mana is larger than or equal to card cost.	health is reflected on the screen.	health is reflected on the screen.
		Player mana amount is deducted by the card's listed card cost, updated remaining mana amount is listed on player screen.	Player mana amount is deducted by the card's listed card cost, updated remaining mana amount is listed on player screen.
		Selected card is destroyed and removed from player screen, card is removed from player deck.	Selected card is destroyed and removed from player screen, card is removed from player deck.
	When an action card (any type) (excluding dimension hop card) is clicked and current mana is smaller than card cost.	Player health and Enemy health remain the same. Current mana amount also remains the same.	Player health and Enemy health remain the same. Current mana amount also remains the same.
		Selected card is not destroyed and remains on the player's screen. The card remains in player deck.	Selected card is not destroyed and remains on the player's screen. The card remains in player deck.
	When end turn button is pressed	Player turn switches to enemy. Player is not allowed to make any actions.	Player turn switches to enemy turn. Player is not allowed to make any actions.
On all enemy turns	When enemy mana is higher than or equal to any card cost and enemy deck on hand is not empty	Enemy will play cards until it runs out of cards on hand or runs out of mana.	Enemy will play cards until it runs out of cards on hand or runs out of mana.
	Enemy plays attack type card when enemy mana is higher than or equal to any card	Player health decreases by card's listed amount, updated player health is reflected on the player screen.	Player health decreases by card's listed amount, updated player health is reflected on the player screen.

	Enemy plays heal type card when enemy mana is higher than or equal to any card	Enemy health increases by card's listed amount, updated enemy health is reflected on the player screen.	Enemy health increases by card's listed amount, updated enemy health is reflected on the player screen.
	When enemy mana is lower than every card cost or there are no more cards on enemy hand	Player turn switches back to player. Enemy cannot do any actions.	Player turn switches back to player. Enemy cannot do any actions.
On all player turns after scene load cycle	Player turn switches from enemy to player	Player hand will increase by 3 cards, the new random card is displayed on player screen.	Player hand will increase by 3 cards, the new random card is displayed on player screen.
		Player current mana resets to 10.	Player current mana resets to 10.
On all enemy turns after scene load cycle	Player turn switches from player to enemy	Enemy hand will increase by 3 cards.	Enemy hand will increase by 3 cards.
		Enemy current mana resets to 10.	Enemy current mana resets to 10.
After Player turn alternates between player and enemy six times (three cycles)	Enemy runs out of mana to play cards or has an empty hand	Game scene will switch from "2D" to "3D".	Game scene will switch from "2D" to "3D".
Game over triggered	Player health less than or equal 0	Game scene will switch to "Game Over" scene.	Game scene will switch to "Game Over" scene.
	Enemy health less than or equal 0	Game scene will switch to "Game Over" scene.	Game scene will switch to "Game Over" scene.

Figure 25: 2D Scene test case

Feature: 3D Fighting Game			
Sub-feature (if any)	Test case	Expected Result	Actual Result

On scene load	Initialize scene	Timer of 30 seconds starts counting down.	Timer of 30 seconds starts counting down.
		Player health and enemy health is the same as the previous 2D scene.	Player health and enemy health is the same as the previous 2D scene.
Player Camera movement control	Move mouse up	Game camera y-axis increases.	Game camera y-axis increases.
	Move mouse down	Game camera y-axis decreases.	Game camera y-axis decreases.
	Move mouse left	Game camera x-axis increases.	Game camera x-axis increases.
	Move mouse right	Game camera x-axis decreases.	Game camera x-axis decreases.
Player movement	Player presses 'W'	Player moves forward.	Player moves forward.
	Player presses 'A'	Player moves left.	Player moves left.
	Player presses 'S'	Player moves backward.	Player moves backward.
	Player presses 'D'	Player moves right.	Player moves right.
Player weapon control	Player presses left mouse button	Player sword swing animation and audio is played.	Player sword swing animation and audio is played.
		If any enemy is within range of sword, enemy takes damage.	If any enemy is within range of sword, enemy takes damage.
Enemy behaviour - The Egg	Enemy pathing	Enemy will always move towards the player.	Enemy will always move towards the player.
	Enemy weapon handling	Enemy will always loop and repeat swing sword action.	Enemy will always loop and repeat swing sword action.
		Sword animation and	Sword animation and

		audio effect plays whenever enemy sword swing action occurs.	audio effect plays whenever enemy sword swing action occurs.
Enemy behaviour - Dragon	Enemy pathing	Enemy randomly patrols around or stay still before detecting the player within the radius. Once the player is detected, the enemy starts chasing. Once the player is close enough, then enemy starts attacking.	Enemy randomly patrols around or stay still before detecting the player within the radius. However, it is possible that the dragon never detects the player due to the random nature of its patrolling state. Once the player is detected, the enemy starts chasing. Once the player is close enough, then enemy starts attacking.
	Enemy weapon handling	The attacking animation is biting and it does it repeatedly until the player is outside of the radius.	The attacking animation is biting and it does it repeatedly until the player is outside of the radius.
Game over triggered	Player health less than or equal 0	Game scene will switch to "Game Over" scene.	Game scene will switch to "Game Over" scene.
	Enemy health less than or equal 0	Game scene will switch to "Game Over" scene.	Game scene will switch to "Game Over" scene.

Figure 26: 3D Scene test case

Feature: Dimension Hopping			
Sub-feature (if any)	Test case	Expected Result	Actual Result
2D card game dimension hop	3 Player Turns and 3 Enemy Turns are	Scene switches from 2D to 3D.	Scene switches from 2D to 3D.

(natural)	completed respectively.		
2D card game dimension hop (accelerated)	Dimension Hop card is clicked.	Scene switches from 2D to 3D.	Scene switches from 2D to 3D.
3D card game dimension hop (natural)	Wait for 30 seconds.	Scene switches from 3D to 2D.	Scene switches from 3D to 2D.

Figure 27: Dimension Hop test case

Feature: Game Over Menu			
Sub-feature (if any)	Test case	Expected Result	Actual Result
On scene load	Play Again button is pressed	Player health and enemy health is reset to 500.	Player health and enemy health is reset to 500.
		2D scene is loaded.	2D scene is loaded.
	Back to Main Menu button is pressed	Main Menu scene is loaded.	Main Menu scene is loaded.

Figure 28: Game Over test case

Flow and Architecture of Gameplay

Flow of Gameplay

1. Starting Screen. The player clicks on 'PLAY'.



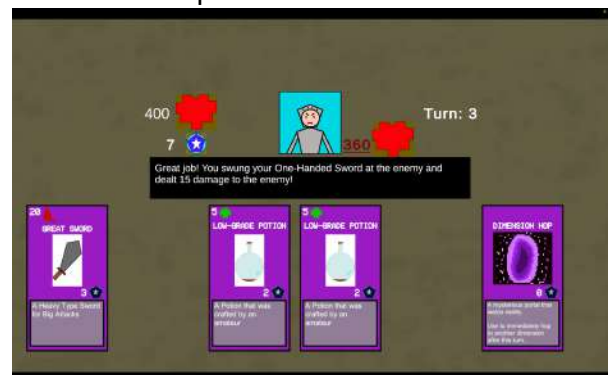
2. The player is loaded to the 2D scene.



3. Player selects cards to deal damage to enemy. When space bar is clicked, the enemy deal damages to the player.



4. Three rounds of attacking back and forth repeat.



5. The player uses Dimension Hop card to switch to 3D, in which the player is immediately loaded to the 3D scene.



6. A dragon immediately attacks the player to protect its egg for 30s. The health of the enemy is preserved from the 2D scene into the 3D scene.



7. The player moves using WASD commands and swings sword using Left Shift key.



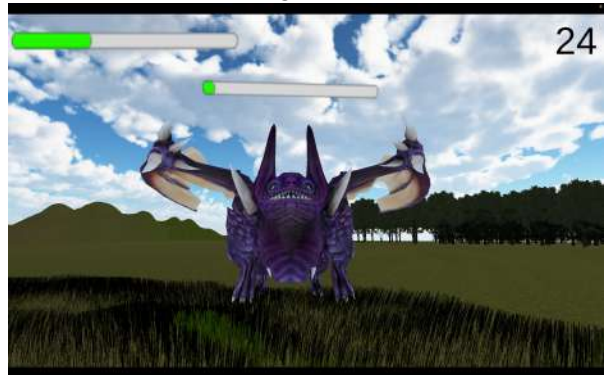
8. The player utilises obstacles to impede dragon's path. The egg also starts chasing the user.



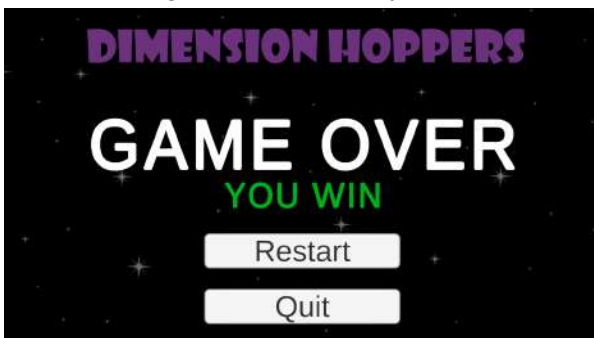
9. After 30 seconds, the player switches back to 2D.



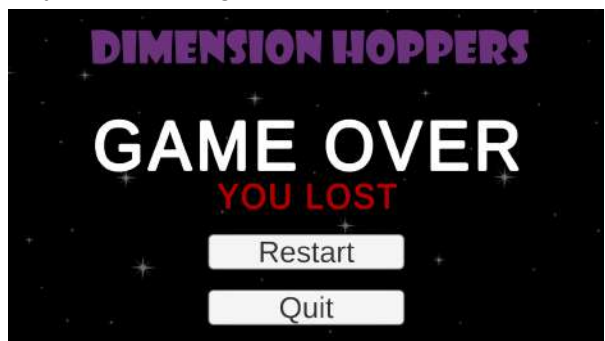
10. The player dimension hops to 3D to attack the dragon.



- 11a. The dragon dies. The player wins.

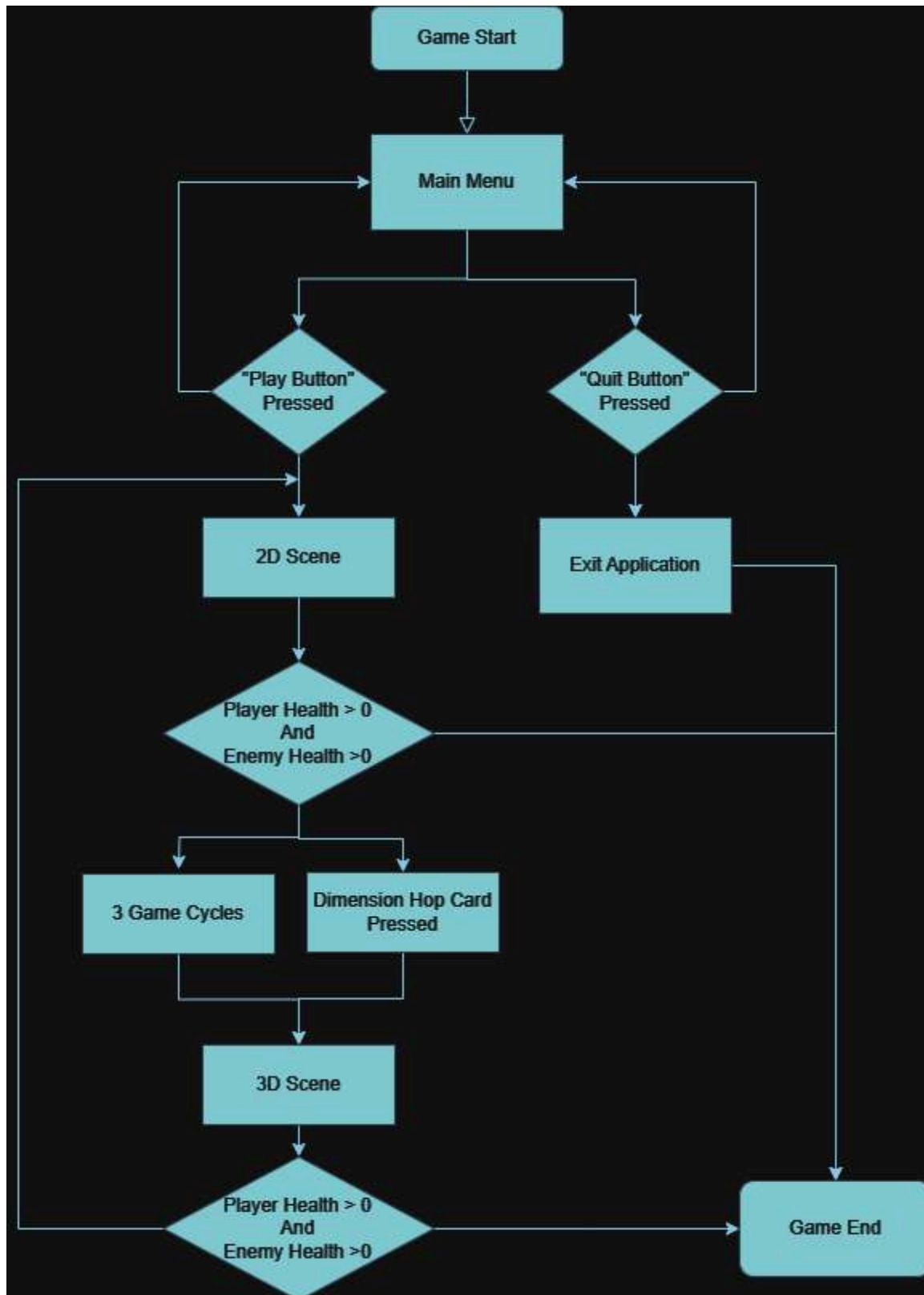


- 11b. If the health of player reaches 0, then the player loses the game.

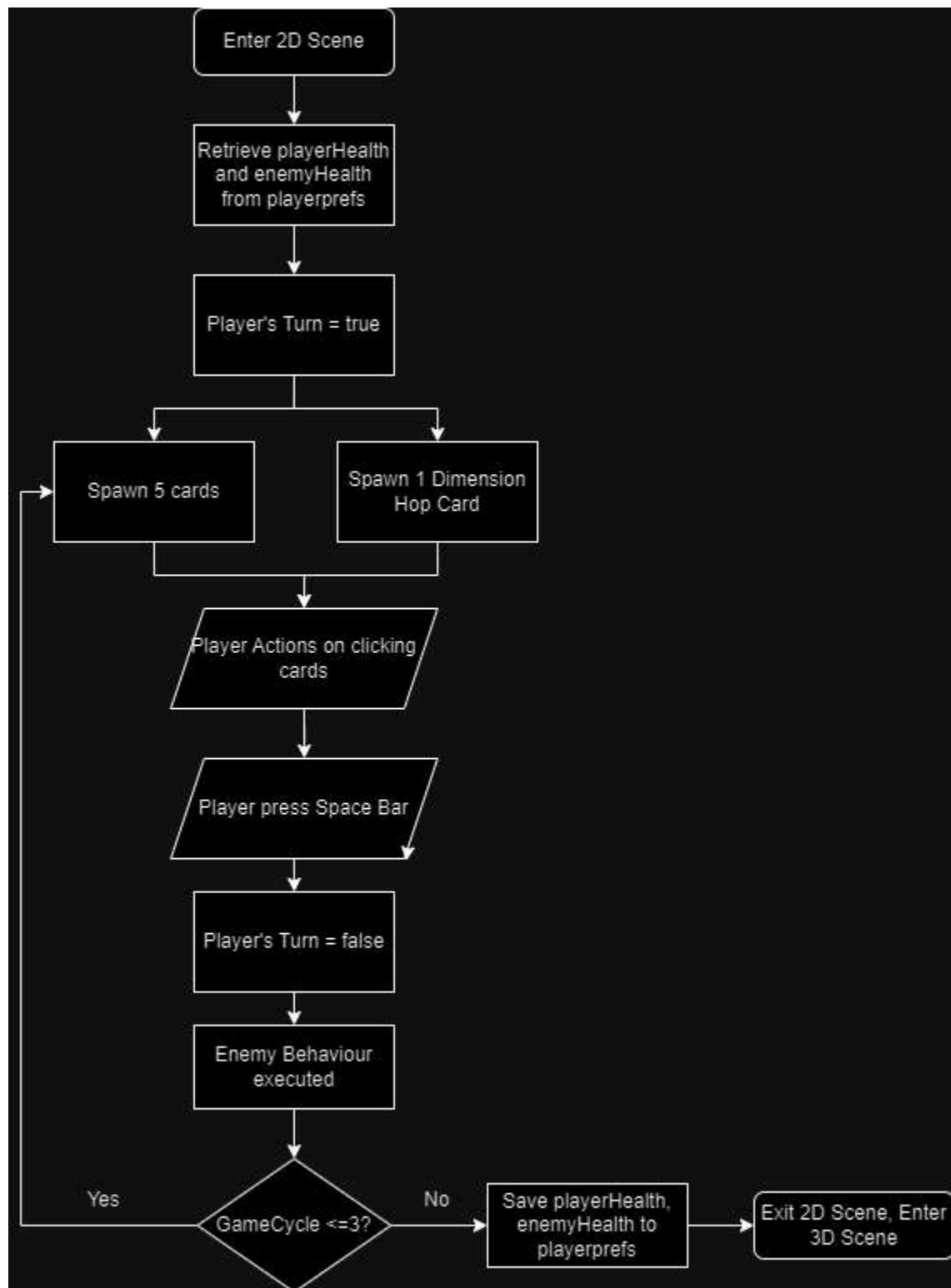


Architecture of Game

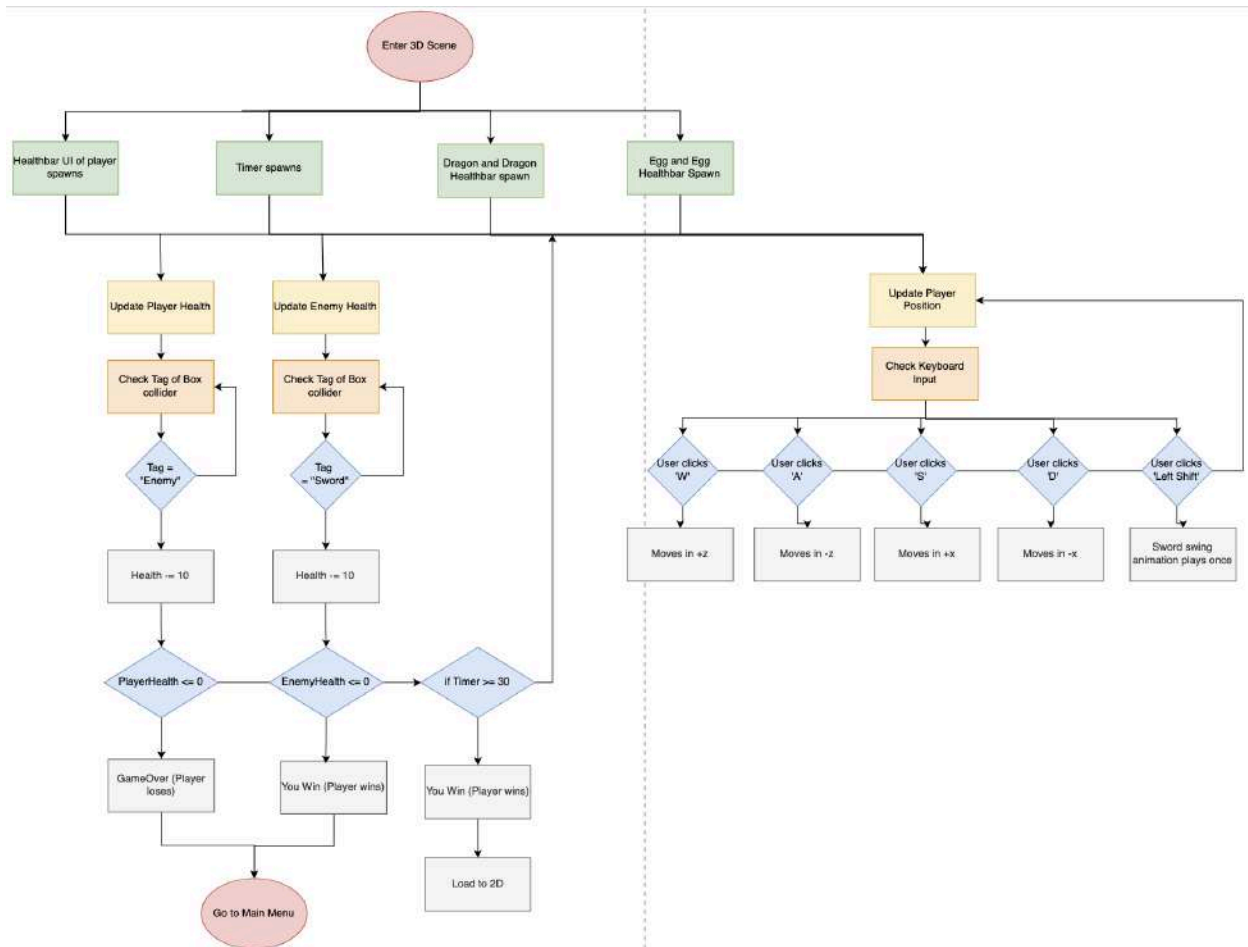
Overall Game Scene Transitioning:



2D Card Game:



3D Card Game:



Timeline of Project

Date	Description
13 May - 20 May	Prepare poster and video for liftoff
20 May - 27 May	Learn Unity and other technical skills
27 May - 3 June	Start building parts of the game in Unity, Prepare Project log and Readme
<u>Milestone 1 (3 June)</u>	Confirmed problem motivation and proposed core features/user stories
3 June - 10 June	Continue building core features within the game
10 June - 17 June	Continue building core features within the game
17 June - 24 June	Complete all core features within the game (2D card game, 3D fighting game, Dimension hopping)
24 June - 1 July	Alpha testing, debugging and start developing side features
<u>Milestone 2 (1 July)</u>	core features fully developed, less problems encountered
1 July - 8 July	Continue developing side features (expanding in-game mechanics) and start beta testing with each newly added side feature
8 July - 15 July	Setting up playerpref system and implement into game
15 July - 22 July	Beta testing and start user documentation
22 July - 29 July	Debugging and finalizing all documentation of the game system
<u>Milestone 3 (29 July)</u>	bugs squashed, edge features developed, problems encountered, and user testing

Figure X: Project Timeline

References

NUS (2024) CS2103/T - Textbook

<https://nus-cs2103-ay2324s2.github.io/website/se-book-adapted/index.html>

Dani Krossing (Jul, 2022) Unity for Beginners!

<https://youtube.com/playlist?list=PL0eyrZgxdwhwQZ9zPUC7TnJ-S0KxqGlrN&feature=shared>

Brackeys (Jan, 2018) SCRIPTABLE OBJECTS in Unity

<https://youtu.be/aPXvoWVabPY?si=Pq7h7BWWU1CDiGUy>

Natty GameDev (Dec, 2021) #1 FPS Movement: Let's Make a First Person Game in Unity!

<www.youtube.com/watch?v=rJqP5EesxLk&list=PLGUw8UNswJEOv8c5ZcoHarbON6m>

BMo (Mar, 2021) How to Make A Simple HEALTH SYSTEM in Unity

<https://www.youtube.com/watch?v=vNL4WYgvwd8&list=PLH6O-D3JQT687xKXE05Gmy283mHwMMD9L>

GDTitans (Jan, 2022) 3D ENEMY AI in UNITY - (E01): STATE MACHINE BEHAVIORS

<https://www.youtube.com/watch?v=whkC8f3oNOk>

Zyger (Dec 2021) Melee Combat & Weapon System - Unity Beginner Tutorial

<www.youtube.com/watch?v=aNZw588BQBo>

2024 Unity Technologies (Jun, 2024) Unity - Manual: Script Serialization

<docs.unity3d.com/Manual/script-Serialization.html>

2024 Unity Technologies (Jun, 2024) Unity - Scripting API: Quaternion.Euler

<docs.unity3d.com/ScriptReference/Quaternion.Euler.html> .

2024 Unity Technologies (Jun, 2024) Unity - Scripting API: NavMeshAgent.stoppingDistance

<docs.unity3d.com/540/Documentation/ScriptReference/NavMeshAgent-stoppingDistance.html>